Day 30 Assignment

1.Basic Questions

1. What is OS Command Injection?

It is a security problem where an application uses user input in an operating-system command, allowing the command to be changed.

2. What is a shell?

A shell is a program that reads commands and asks the operating system to run them.

3. Why is putting user input into a shell command dangerous?

Special characters in the input may be treated as commands instead of normal text.

4. What does whoami do?

It displays the username of the account running the command.

5. What is server-side validation?

It means checking data on the server before the application uses it.

6. What is allowlist validation?

It accepts only values that are specifically allowed and rejects other values.

7. What does Least Privilege mean?

An application should have only the permissions it needs to do its job.

8. Why are safer platform APIs preferred?

They can perform tasks without using a shell, reducing the risk of command injection.

9. SQL Injection vs OS Command Injection?

SQL Injection targets database queries. OS Command Injection targets commands run by the operating system.

10. XSS vs OS Command Injection?

XSS affects content running in a user's browser. OS Command Injection affects command execution on the server.

2. PortSwigger Practice Lab

1. Which feature was vulnerable?

The stock-check feature was vulnerable because user-controlled input affected an operating-system command.

2. Which request parameters were present?

The request included product or store-related parameters. Use the exact names from your own captured lab request.

3. Which parameter was changed?

The Store ID parameter was changed during the lab exercise.

4. What test value was used?

A controlled test value was used in the authorized PortSwigger training lab. Record the exact value from your own request.

5. Which harmless command was used?

The command was whoami.

6. What does the command show?

It shows the operating-system username running the application process.

7. Username returned by the lab:

Peter-PeRelc

8. Which Burp Suite feature was used?

Burp Suite Repeater was used to resend the request and inspect the response.

9. Did the lab show “Solved”?

Yes

10. Screenshot evidence



3. How to Make the Application Safer

1. What is wrong with the design?

It places a user-controlled Store ID into a shell command. This may let input change what the system runs.

2. What is a safer approach?

Use a platform API or library instead of building a shell command. If a shell is unavoidable, keep the command structure fixed and handle arguments safely.

3. If valid Store IDs are 1, 2 and 3, what validation is needed?

Accept only 1, 2 or 3 on the server. Reject all other values.

4. Why use Least Privilege?

It limits what an application can access or do if an attack succeeds.

5. How should errors be handled?

Keep technical details in secure server logs and show users a general error message. Detailed errors may reveal system information.

4. Comparing Common Vulnerabilities

Vulnerability	Main target	Main protection
SQL Injection	Database queries and records	Use parameterized queries or prepared statements.
Cross-Site Scripting (XSS)	Content and scripts in a user's browser	Encode output correctly and use safe DOM methods; CSP adds another layer.
OS Command Injection	Operating-system commands on the server	Avoid shell commands, validate input with allowlists and use least privilege.

5. What I Learned

This lab helped me understand that user input should be treated as data, not as a command. If an application adds untrusted input directly to a shell command, the system may run something unintended. The whoami command can safely show which account is running a process in a training lab. I learned that server-side allowlists and safer APIs help prevent command injection. Least Privilege can reduce the damage if a weakness is exploited. Applications should keep user input separate from operating-system commands.

Safety note: Use screenshots only from your authorized training lab. Do not include real credentials, cookies or tokens.